

Sorting Algorithms



The Sorting Problem

- Given a list of numbers, reorder them so that they end in ascending order.

lst = [5, 8, 12, 7, 8, 10]

some_sort(lst)

print(lst) # [5, 7, 8, 8, 10, 12]

Selection Sort

[5, 8, 12, 7, 8, 10]

[5, 7, 12, 8, 8, 10]

[5, 7, 8, 12, 8, 10]

[5, 7, 8, 8, 12, 10]

[5, 7, 8, 8, 10, 12]

$$\begin{aligned} T(n) &= \sum_{curr=1}^n \Theta(n) \\ &= 1 + 2 + 3 + 4 + 5 + \dots + n \\ &= \frac{n(n-1)}{2} = \frac{n^2 - n}{2} \\ &= \boxed{\Theta(n^2)} \end{aligned}$$

```
def swap(lst, curr, min_idx):  
    temp = lst[curr] #  $\Theta(1)$   
    lst[curr] = lst[min_idx] #  $\Theta(1)$   
    lst[min_idx] = temp #  $\Theta(1)$ 
```

```
def selection_sort(lst):  
    n = len(lst) #  $\Theta(1)$   
    #  $\Theta(n)$   
    for curr in range(n):  
        min_idx = curr #  $\Theta(1)$   
        #  $\Theta(curr) \leq \Theta(n)$   
        for j in range(curr+1, n):  
            if lst[j] < lst[min_idx]: #  $\Theta(1)$   
                min_idx = j #  $\Theta(1)$   
        swap(lst, curr, min_idx) #  $\Theta(1)$ 
```

Short Aside... what's the asymptotic runtime of the following function?

```
def fun(n):
```

```
    if n == 0:
```

```
        print("Done")
```

```
    else:
```

```
        for i in range(7):
```

```
            print(n)
```

```
        fun(n//2)
```

$\Theta(1)$

$\Theta(1)$

$\Theta(1)$

$\Theta(\log_2 n)$

FIRST PASS

[5, 8, 4, 1, 9, 3, 0, 1]

[5, 6, 8, 1, 9, 3, 0, 1]

[5, 6, 1, 8, 9, 3, 0, 1]

[5, 6, 1, 8, 3, 9, 0, 1]

[5, 6, 1, 8, 3, 0, 9, 1]

[5, 6, 1, 8, 3, 0, 1, 9]

THIRD PASS

[1, 5, 6, 3, 0, 1, 8, 9]

[1, 5, 3, 6, 0, 1, 8, 9]

[1, 5, 3, 0, 6, 1, 8, 9]

[1, 5, 3, 0, 1, 6, 8, 9]

SECOND PASS

[5, 1, 6, 9, 3, 0, 1, 9]

[5, 1, 6, 3, 8, 0, 1, 9]

[5, 1, 6, 3, 0, 8, 1, 9]

[5, 1, 6, 3, 0, 1, 8, 9]

```
def bubble_sort(lst):
```

```
    n = len(lst) #  $\Theta(1)$ 
```

```
    #  $\Theta(n)$ 
```

```
    for i in range(n-1):
```

```
        for j in range(n-i-1):
```

```
            if lst[j] > lst[j+1]:
```

```
                lst[j], lst[j+1] = lst[j+1], lst[j]
```

$\Theta(n)$

↑

$\Theta(i)$

$\Theta(1)$

$\Theta(1)$

$\Theta(1)$

$$T(n) \approx \sum_{i=1}^n i$$

$$= 1 + 2 + 3 + 4 + 5 + \dots + n$$

$$= n(n-1)/2 = n^2/2 - 1/2$$

$$= \Theta(n^2)$$

$\Omega(n)$

$\Theta(n^2)$

idx: 7

[5 8 6 1 9 3 0 2]

[5 6 8 1 9 3 0 2]

[5 6 1 8 9 3 0 2]

[5 1 6 8 9 3 0 2]

[1 5 6 8 9 3 0 2]

[1 5 6 8 3 9 0 2]

[1 5 6 3 8 9 0 2]

[1 5 3 6 8 9 0 2]

[1 3 5 6 8 9 0 2]

[0 1 3 5 6 8 9 2]

[0 1 2 3 5 6 8 9]

def insertion_sort(lst):

for idx in range(1, len(lst)):

curr = lst[idx]

j = idx

while j >= 1 and lst[j-1] > curr:

lst[j] = lst[j-1]

j -= 1

lst[j] = curr

$\Theta(n)$

$\Theta(1)$

$\Theta(1)$

$\Theta(idx) = \Theta(n)$

$\Theta(1)$

$\Theta(1)$

$\Theta(1)$

$$T(n) = \sum_{idx=1}^n idx$$

$$= 1 + 2 + 3 + 4 + \dots + n$$

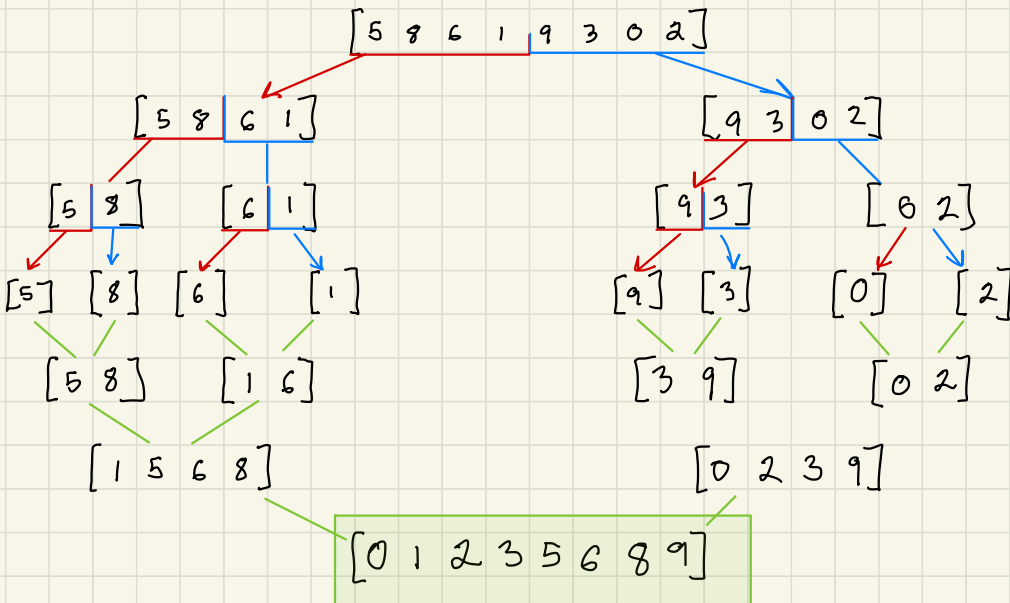
$$= \frac{n^2 - n}{2}$$

$$= \Theta(n^2)$$

$\Omega(n)$ for already sorted

$$\mathcal{O}(n^2)$$

Merge Sort



```
def merge(srt_lst1, srt_lst2):
    merged_list = [] # 0(1)
    idx_1 = 0 # 0(1)
    idx_2 = 0 # 0(1)

    # 0(n)
    while idx_1 < len(srt_lst1) and idx_2 < len(srt_lst2):
        if srt_lst1[idx_1] < srt_lst2[idx_2]: # 0(1)
            merged_list.append(srt_lst1[idx_1]) # 0(1)
            idx_1 += 1 # 0(1)
        else:
            merged_list.append(srt_lst2[idx_2]) # 0(1)
            idx_2 += 1 # 0(1)

    # 0(n)
    while idx_1 < len(srt_lst1): # 0(1)
        merged_list.append(srt_lst1[idx_1]) # 0(1)
        idx_1 += 1 # 0(1)

    # 0(n)
    while idx_2 < len(srt_lst2): # 0(1)
        merged_list.append(srt_lst2[idx_2]) # 0(1)
        idx_2 += 1 # 0(1)

    return merged_list # 0(1)
```

```
def merge_sort(lst):
    if len(lst) == 0: # 0(1)
        return # 0(1)
    elif len(lst) == 1: # 0(1)
        return # 0(1)
    else:
        mid = len(lst) // 2 # 0(1)
        left_lst = lst[:mid] # 0(n)
        right_lst = lst[mid:] # 0(n)

        merge_sort(left_lst)
        merge_sort(right_lst)

        merged = merge(left_lst, right_lst) # 0(n)

        for i in range(len(merged)):
            lst[i] = merged[i] # 0(n)
```

$$T(n) = \Theta(n) + \Theta(n) + \Theta(n) + \dots + \Theta(n)_{\log n} = \boxed{\Theta(n \log n)}$$